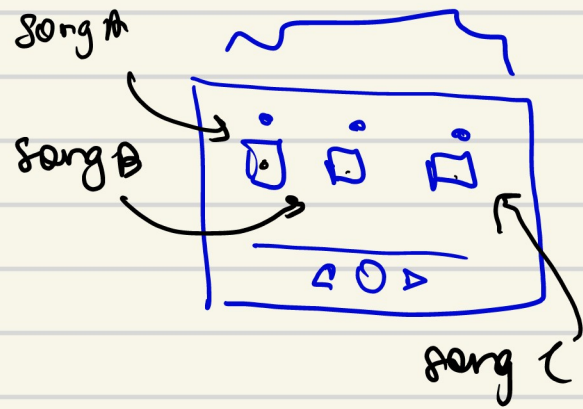


JS - interactive
 React → library
 → META
 → sensibility

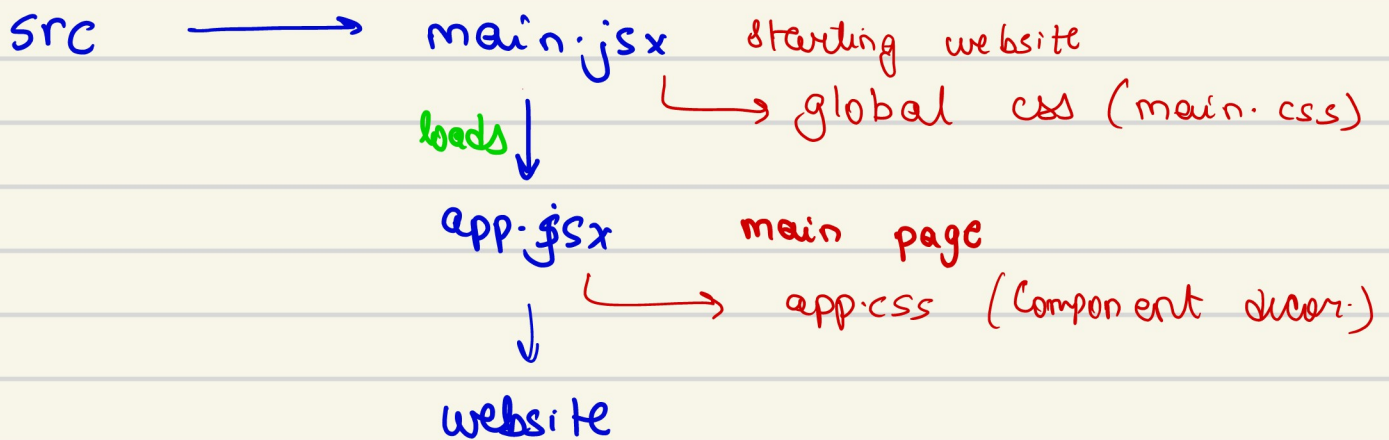
✓ Song function



CSS: Tailwind :: JS: React

* NPM = node package manager
 ↓ ↓ ↓
 React vite Node JS
 = App Store of libraries

Vite → fast dev. server → project structure
 → hot reloads
 npm create vite @latest



package.json — recipe book
 versions of packages

main.jsx	app.jsx	app.css	index.css
Main gate	Festival ground	ground decor.	rules for entire ground

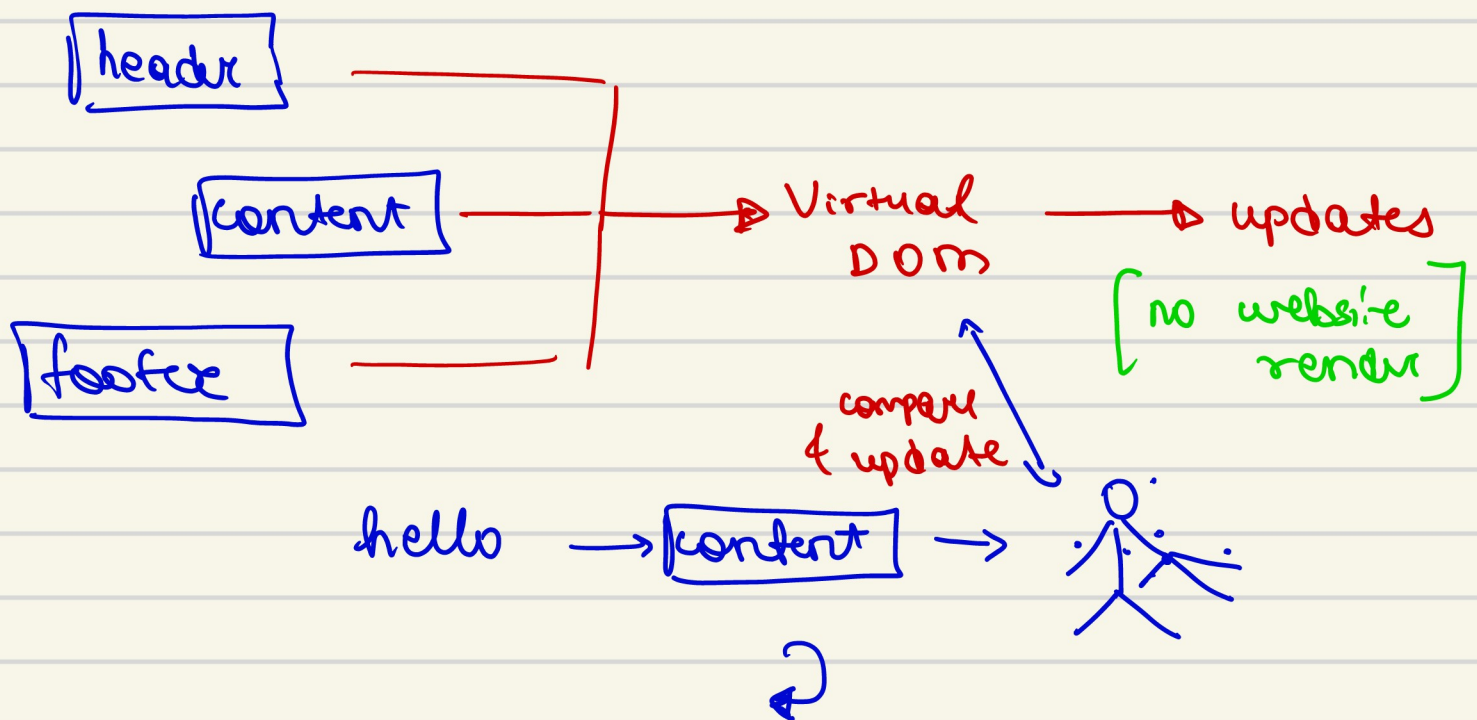


Created with
 NoteWise

★ React Libraries

fetch data - Tanstack
build locally - Vite
Component UI - ShadCN
api calls - Axios
state management - React

Vanilla JS classic
React VDOM
Preact lightweight VDOM
Vue.JS Optimized DOM
Lit shadow DOM



- ① VDOM - no crash
- ② Reusability
- ③ Flexibility

* Show / Hide an element

① Vanilla JS

```
const toggleBtn = document.getElementById('toggle-btn');  
const box = ''
```

grab elements from DOM

```
let isVisible = true;
```

```
toggleBtn.addEventListener('click', () => {
```

```
  isVisible = !isVisible
```

↳ turns to FALSE

```
    if (isVisible)
```

```
    {
```

```
      box.classList.remove('hidden');
```

```
    }
```

```
    else
```

```
    {
```

```
      box.classList.add('hidden');
```

```
    }
```

```
  }  
});
```

VANILLA



REACT

function [greet()]

function ToggleBox() → Component

state const [isVisible, setIsVisible] = useState(true)
true/false fn. ↳ understands when someth. is changed
updates value if called

return (

<div>

<button onClick = { () =>

setIsVisible(!isVisible) } > Toggle </button>

it directly triggers state change on click

<div className = { p-4 bg-white

{ isVisible ? '' : 'hidden' }

> Surprise! </div> ↳ define an expr.

}) ;

① Button Click → setIsVisible() → State changes
(false)

Virtual DOM updated ← New JSX is created ← Component runs again

↳ React finds difference → Update that

* Dark mode .

② Weather

③ Node JS

④ Express JS



① Components —

- building blocks of React
- reused only
- JS → JSX
JSXML

<header />
<footer />
<content />

JS

React · create Element (
 "h1",
 null,
 "Hello"
);

React

<h1> Hello </h1>
↓

- Camel Case ✓✓

<input maxLength = "10"> // <input maxLength
= {10}>
onClick // onClick
↳ by default



② Props

used to pass an argument into a fn.

- component
- string
- function

JS

```
function greet(name)
{
  return 'Hello $ {name}';
}

greet('Atharva');
```

↳ call fn.

OUTPUT = Hello Atharva

React

```
<Greeting text = {Hi} />
function Greeting(props)
{
  return <h1> {props.text}
  </h1>
}
```

props = property

props = {

```
  children1: [
    <h1> ... </h1>
    <p> ... </p>
  ],
  children2: [
    <h1> ... </h1>
    <p> ... </p>
  ]
}
```

```
function card(props)
{
  return (
    <div>
      {props.children1}
    </div>
  );
}
```

Key Prop

- unique numbers/string
- stored in lists
- diff. in react component


```

{
  items.map (item => (
    <component key = {item.id} />
  ));
}

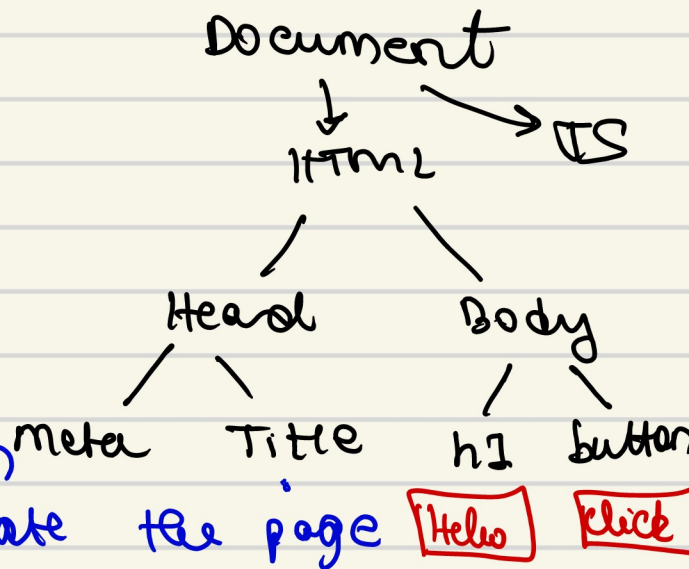
```

items.map

(id = 2)

③ Render -

- browser is dumb
- HTML or JS → JSX
[stores it like a tree]



- find element → change DOM
↳ update the page
- VDOM stores a copy of the JS object in memory / RAM.
↳ updates only element

④ Event handling -

JS

```
const btn = document.getElementById("btn");  
btn.addEventListener(  
  "click" () => {  
    console.log("clicked");  
  }  
);
```

finds element
↓
adds an event listener
↓
run function

React

event handler

```
<button onClick = { () => {  
  console.log("clicked");  
}} > Click me </button>
```

attaches event
↓
runs fn.

- onSubmit
- onClick

- onChange
- onBlur

- onFocus
- onHover

⑤ Hooks -

```
function add(a, b)  
{  
  return a + b;  
}
```

reuse } Hooks

functions that gives
react to features to
a partic. component



① use State - state man. } runs 1 time

② use Effect - gives effects

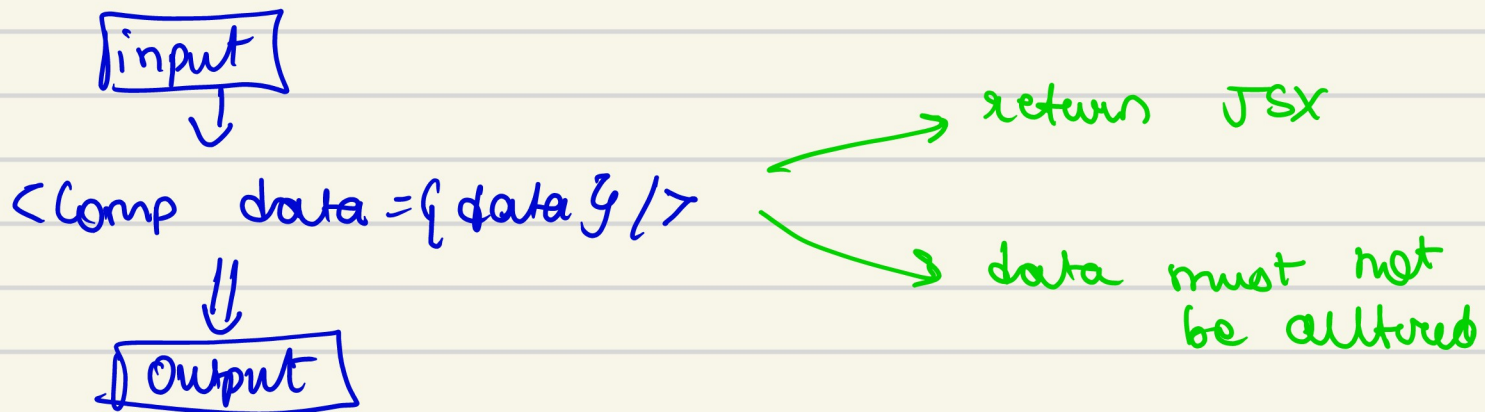
```
useEffect(() => {  
  console.log("Happy");  
}, []);
```

} runs only 1

③ use Reducers ④ useRef ⑤ use Memo

⑥ use Callback

⑥ Purity - how react comp. should work



let count = 0;

```
function drawya()  
{
```

```
  count = count + 1
```

```
  return <h2>drawya{count}</h2>  
}
```



⑦ Strict Mode To prevent errors

< Strict mode >

< App / >

.....

< / Strict mode >

} → error
→ forcefully ignore it
→ as it is

⑧ Content -

< App / >

add prop

< Header / >

add prop

saves memory
spends time

Memory Space $\propto \frac{1}{\text{Time}}$

< Title >

add prop

⑨ Suspense -

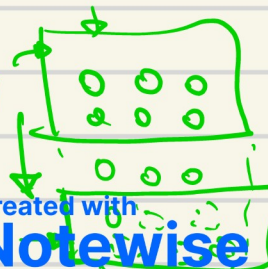
↻ ↻ ↻ ↻

component to wait for something

→ UX is slow

→ lazy loading

[loads specific parts on demand]



Created with Notewise